Ders 14 Genişletilmiş Notu: Ayrık Zincirleme, Evrensel Hashing ve Yük Faktörüne Göre Arama Maliyeti

Ders 14 Genişletilmiş Notu: Ayrık Zincirleme, Evrensel Hashing ve Yük Faktörüne Göre Arama Maliyeti

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Ayrık Zincirleme (Separate Chaining)
- 2. Ayrık Zincirleme Ne Zaman Kullanılır, Karmaşıklığı Nedir?
- 3. Ayrık Zincirlemenin C ile Gerçeklenmesi
- 4. Evrensel Hashing (Universal Hashing)
- 5. Yük Faktörü ve Ortalama Sınama Sayısı
- 6. Başarısız Aramada Ortalama Adım Sayısının Türetilmesi
- 7. Doğrusal Sınamada Başarılı ve Başarısız Arama
- 8. Ayrık Zincirlemede Arama Maliyeti
- 9. Doğrusal ve İkinci Dereceden Sınamanın Karşılaştırılması
- 10. Ders Sonu Soruları

1. Ayrık Zincirleme (Separate Chaining)

Bu kayıt, günün ikinci oturumudur. Hoca derse, bir önceki oturumda açık adreslemeyi (open addressing) tamamladıklarını hatırlatarak başladı: doğrusal sınama (linear probing), ikinci dereceden sınama (quadratic probing) ve çift hashing (double hashing) konuşulmuştu. Sırada çakışma çözümünün öteki ana yolu olan ayrık zincirleme (separate chaining) vardı.

Ayrık zincirlemede tablonun her gözü bir bağlı listenin (linked list) başını tutar. Bir anahtarın adresine daha önce başka bir eleman yerleşmişse yeni eleman o gözün bağlı listesine eklenir. Listenin son düğümü NULL gösterir. Böylece listenin bittiği anlaşılır.

1.1 Tahtadaki örnek: $m = 6$

Hoca tablo boyutunu $m = 6$ aldı ve hash fonksiyonu olarak $h(k) = k$ mod 6 kullandı.

- 13 geldi: $13 \bmod 6 = 1$. 13, 1 numaralı göze yerleşti.
- 1 geldi: $1 \bmod 6 = 1$. Bu da 1 numaralı göze düştü.
- Sonra hoca sordu: 19 geldiğinde nereye yerleşir? $19 \bmod 6 = 1$ olduğu için 19 da 1 numaralı gözün listesine girer.

Soru: 19 geldiğinde nereye yerleşir? **Cevap (öğrenci):** 13'ün yanına, sağına. Hoca "13'ün ne tarafına?" diye sorunca öğrenci önce "sağına" dedi, sonra sıralama yapılmıyorsa listenin başına eklemenin daha mantıklı olacağını söyledi. Hoca da bu cevabı beklediğini söyledi.

Hocanın vardığı sonuç şudur: Yeni elemanı bağlı listenin sonuna eklemek için listenin sonuna kadar gitmek gerekir ve bu gereksiz bir maliyettir. Adres doluysa yeni elemanı listenin **başına** eklemek çok daha akıllıcadır.

> **Hocanın vurgusu:** Ayrık zincirleme yerleştirmesinde bağlı listenin sonuna eklemek yerine başına eklemek daha akıllıcadır. Buna dikkat etmekte fayda var.

1.2 Tahtadaki bilerek yapılmış hata

Hoca tahtadaki çizimde bir hata olduğunu ve kimsenin bunu dile getirmediğini söyledi. Bir öğrenci, yerleştirmenin üçüncü gözden değil birinci gözden başlaması gerektiğini söyledi. Hoca "aynen öyle, bunu bekliyordum" dedi ve çizimi düzeltti: 19, 3 numaralı göze değil, 1 numaralı gözün listesine girer.

2. Ayrık Zincirleme Ne Zaman Kullanılır, Karmaşıklığı Nedir?

2.1 Tablo boyutu ile eleman sayısı ilişkisi

Soru: Ayrık zincirlemeyi, tablo boyutu m ile eleman sayısı n arasında nasıl bir ilişki varken kullanmak akıllıca olur? **Cevap:** $n > m$ iken. Eleman sayısı tablo boyutundan büyükse açık adresleme yerine ayrık zincirleme tercih edilmelidir.

Açık adreslemede her göze tek eleman girer. Bu yüzden $n > m$ iken tablo dolar. Ayrık zincirlemede ise bir göze istenen kadar eleman bağlanabilir.

2.2 Karmaşıklık: teoride $O(n)$, pratikte $O(1)$

Soru: Ayrık zincirleme kullanan bir hashleme yaklaşımının karmaşıklığı nedir? **Cevap (öğrenci):** En kötü durumda bütün elemanlar aynı adrese düşerse $O(n)$ denebilir.

Hoca bunun teorik olarak doğru olduğunu söyledi. Gerçekten bütün elemanlar aynı adrese gelirse karmaşıklık $O(n)$'e çıkar. Fakat pratikte bütün elemanların aynı göze düşmesi için ya çok kötü bir hash fonksiyonu ya da çok kötü bir tablo boyutu seçilmiş olmalıdır. Bu yüzden pratikte ayrık zincirleme de $O(1)$ olarak geçer. Hoca bunun çeşitli istatistiksel ve formül ispatları olduğunu söyledi.

Hocanın açıklaması şöyleydi: Bir elemana 10 aramada da ulaşabilirsiniz, 50 aramada da. Önemli olan, bu sayının n'e bağlı olmamasıdır. Hoca örnek olarak "500 elemanınız var, 50. adımda ulaştınız; 5000 elemanınız var, 90. adımda ulaştınız; bunların hepsi $O(1)$ düzeyinde kabul ediliyor" dedi.

> **Ders sonrası not:** Hocanın örneğinde adım sayısı eleman sayısıyla birlikte 50'den 90'a çıkıyor. Adım sayısı n ile büyüyorsa bu $O(1)$ değildir. Doğru ifade şudur: Yük faktörü $\alpha = n/m$ sabit tutulursa (eleman sayısı arttıkça tablo da büyütülürse) ortalama arama maliyeti $\Theta(1 + \alpha) = O(1)$ olur. Bölüm 8'deki formüller bunu gösterir.

3. Ayrık Zincirlemenin C ile Gerçeklenmesi

Ayrık zincirlemede üç temel işlem vardır: ekleme (insert), silme (delete) ve arama (search). Hoca bunları gerçeklemek için `struct` yapılarına ihtiyaç olduğunu söyledi ve kodu öğrencilerle birlikte, soru sorarak yazdı.

3.1 `struct node`

Soru: Her eleman bir düğüme denk gelecek. Bu düğümde hangi bilgileri saklamalıyım? **Cevap (öğrenci):** Elemanın kendi değerini ve bir sonraki düğümün adresini.

Hoca "kendi değeri" olarak `int key` alınabileceğini söyledi. Yukarıdaki örnekte 13, 1 ve 19 birer anahtardır. Ayrıca anahtara karşılık gelen bir değer (value) tutulur. Hoca bunun için `name` alanını kullandı. Son olarak, öğrencinin söylediği gibi, sonraki düğümün işaretçisi (pointer) gerekir.

Hoca burada `typedef` kullanmadığını, `struct node` yazdığını hatırlattı. Yapının içinde kendi türüne bir işaretçi olduğu için `typedef` ile tanımlanan kısa adı yapının içinde kullanma şansı yoktur.

3.2 `struct hash`

Soru: Bu durumda `struct hash` neyi tutmalı? **Cevap (öğrenci):** Her gözde bir liste başı ve o gözde kaç düğüm olduğu.

Hoca `struct node *head` ve `int count` alanlarını yazdı. Öğrenci aynı şeyi dizi olarak tutmayı düşündüğünü söyledi, hoca bunun da olabileceğini kabul etti. Tablonun kendisi bu yapıdan oluşan bir dizidir.

Kod, kısa nottaki tahta karelerinden alınmıştır; bu işte kare olmadığı için altyazıyla yalnız kısmen doğrulanabildi:

```
struct node {
    int key;
    char name[40];
    struct node *next;
};

struct hash {
    struct node *head;
    int count;
};

struct hash *hashTable = NULL;
```

`hashTable`, `struct hash` tipinde bir işaretçidir ve başlangıçta NULL değerini alır.

3.3 `main` fonksiyonu

Hoca `main` içinde bir `key` ve bir `name` tanımladı. Sonra tabloya M göz için yer açtı ve yer açılıp açılamadığını kontrol etti. Yer açılamadıysa bir mesaj verip programdan çıktı. Ardından üç işlem çağrıldı: `insertToHash` anahtarı ve adı alır, `searchInHash` ve `deleteFromHash` yalnız anahtarı alır.

```
hashTable = (struct hash *) calloc(M, sizeof(struct hash));
if (hashTable == NULL) {
    printf("Yer acilamadi");
    exit(0);
}
insertToHash(key, name);
searchInHash(key);
deleteFromHash(key);
```

> **Ders sonrası not:** Yer açılamadığında `exit(0)` yerine hata kodu (`exit(1)` ya da `EXIT_FAILURE`) döndürmek daha doğrudur. Burada `calloc` kullanılması ise iyi bir seçimdir: Bütün `head` alanları NULL, bütün `count` alanları 0 olur. `malloc` kullanılsaydı `head` alanları ayrıca NULL yapılmalıydı; yoksa eklemedeki `if (!head)` kontrolü çöp değer okurdu.

3.4 `insertToHash`

Soru: Ekleme yaparken hangi adımlardan geçmem gerekiyor? **Cevap (öğrenci):** Önce M'ye göre mod alacağız, yani önce hash indeksini hesaplayacağız.

Fonksiyona dışarıdan alınan anahtar ve bu anahtara ait değer (ad) aktarılır. M fonksiyonun içinde de tanımlanabilir, parametre olarak da gönderilebilir; hoca bunun fark etmediğini söyledi. İlk satırda `hashIndex = key % M` hesaplanır.

Sonra hoca, bağlı listelerde genelde yapıldığı gibi, `createNode` fonksiyonunu çağırıp yeni düğüm için yer açtı. Ardından `hashTable[hashIndex].head` kontrolünü yazdı.

Soru: Bu kontrol neye bakıyor? **Cevap (öğrenci):** O hash indeksinin daha önce doldurulup doldurulmadığına.

Yani tablodaki ilgili göz boş mu dolu mu diye `head`'e bakılır. Göz boşsa `head`'e yeni düğüm atanır, `count` 1 yapılır ve fonksiyondan dönülür. Eleman doğrudan ana tabloya yerleştiyse iş biter.

Soru: Göz doluysa ne yapacağız? **Cevap (öğrenci):** O gözün bağlı listesine ekleme yapacağız. Başa ekliyorsak yeni düğüm doğrudan listenin başı olur. Sona ekliyorsak `next`, `next`, … diye listenin sonuna kadar gidip oraya ekleriz.

Hoca başa eklemeyi seçti: Yeni düğümün `next`'i eski başı gösterir, `head` yeni düğüm olur ve `count` bir artırılır.

```
void insertToHash(int key, char name[]) {
    int hashIndex = key % M;
    struct node *newNode = createNode(key, name);
    if (!hashTable[hashIndex].head) {
        hashTable[hashIndex].head = newNode;
        hashTable[hashIndex].count = 1;
        return;
    }
    newNode->next = hashTable[hashIndex].head;
    hashTable[hashIndex].head = newNode;
    hashTable[hashIndex].count++;
}
```

Kısa nota göre tahtadaki kod büyük olasılıkla `count` satırında yarım kalmıştır. Son satır, hocanın "count'u bir artıracağım" sözüne göre tamamlanmıştır.

3.5 Sona ekleme gerekseydi: `tail`

Soru: Sona eklemeli bir yapı olsaydı hangi bilgiyi saklamak akıllıca olurdu? **Cevap (öğrenci):** İleriyi ve geriyi saklayarak, yani çift bağlı liste (doubly linked list) ile.

Hoca bunun da olabileceğini, ama soruyu tek bağlı liste (singly linked list) için sorduğunu söyledi. Öğrencinin ikinci önerisi `head` ile birlikte global bir "current" düğüm tutmaktı. Hoca bunu düzeltti: Global bir current değil, listenin son elemanının adresi, yani `tail` saklanmalıdır. `tail` bilinirse listenin sonuna doğrudan eklenebilir.

3.6 `createNode`

`createNode` fonksiyonuna anahtar ve karakter dizisi aktarılır. Karakter dizisi işaretçi ile karşılanabilir. Fonksiyon `malloc` ile bir düğüm için yer açar, anahtarı ve adı kopyalar, `next`'i NULL yapar ve yeni düğümü döndürür. Hoca yazarken önce bir örnek olarak "age" alanına değer koydu, sonra bu değeri fonksiyona aktarmadıklarını söyleyip `name` kopyalamasıyla düzeltti.

```
struct node *createNode(int key, char *name) {
    struct node *newNode = (struct node *) malloc(sizeof(struct node));
    newNode->key = key;
    strcpy(newNode->name, name);
    newNode->next = NULL;
    return newNode;
}
```

> **Ders sonrası not:** Kısa nota göre tahtada büyük olasılıkla `size(struct node)` yazılmıştı; doğrusu `sizeof(struct node)`. Ayrıca `strcpy`, 40 karakterden uzun bir adda dizinin dışına taşar; `strncpy` ya da uzunluk kontrolü kullanılmalıdır.

Soru: `key % M` satırında M'yi hiç tanımlamadık. Bu bir değişken mi olacak? **Cevap:** M global bir değişken olarak kabul edilebilir ya da fonksiyona `int M` diye parametre olarak verilebilir.

3.7 `searchInHash`

Arama fonksiyonu yalnız anahtarı alır. Hoca bu fonksiyonun kodunu uzun uzun yazmadı, çünkü yapısı bağlı listede aramayla aynıdır. Adımlar öğrencilerle birlikte şöyle kuruldu:

1. Eklemedeki gibi `hashIndex = key % M` hesaplanır.

2. Tablonun o indeksine gidilir ve `head`'den başlanır.
3. İki koşul birlikte sağlandıkça `next` ile ilerlenir: Düğüm NULL değildir ve düğümdeki anahtar aranan anahtara eşit değildir.
4. Döngüden çıkınca, eleman bulunduğu için mi yoksa liste bittiği için mi çıkıldığı kontrol edilmelidir. Bunun için bir bayrak (flag) tutulur. Eleman bulununca bayrak set edilir.
5. Bayrak set edildiyse eleman bulunmuştur. Set edilmediyse "aradığınız eleman bulunamadı" mesajı döndürülür.

Hoca silme (delete) kısmını öğrencilerin düşünmesine bıraktı ve bu kodları yazmalarının iyi olacağını söyledi.

Soru: Açık adreslemenin ayrık zincirlemeye göre bir avantajı var mı, hangi durumda hangisi kullanılmalı? **Cevap:** Hoca bunu daha önce konuştuklarını hatırlattı. Bir öğrenci cevabı verdi: n, m'den büyükse (eleman sayısı fazlaysa) ayrık zincirleme daha akıllıcadır. Diğer durumda açık adresleme üzerinden ilerlemek tavsiye edilir.

4. Evrensel Hashing (Universal Hashing)

Evrensel hashingde tek bir sabit hash fonksiyonu yoktur. Elimizde birden fazla hash fonksiyonu vardır. Bunu hash fonksiyonlarından oluşan bir küme olarak düşünebiliriz. Bu kümeden bir fonksiyon rastgele seçilir.

Hocanın tahtaya yazdığı aile şudur:

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$$

Burada a, b ve p rastgele seçilir. Değişmeyen ve rastgele olmayan tek şey tablo boyutu m'dir.

4.1 Sayısal örnek

Verilen: $a = 3$, $b = 4$, $p = 17$, $m = 6$, anahtar $k = 8$.

- $ak + b = 3 \cdot 8 + 4 = 28$
- $28 \bmod 17 = 11$
- $11 \bmod 6 = 5$

Sonuç: $h_{3,4}(8) = 5$. Anahtar 5 numaralı göze yerleşir.

4.2 Neden böyle bir yapı?

Soru: a, b, p rastgele seçiliyor, yani hash fonksiyonu rastgele değerler üzerinden oluşuyor. Neden böyle bir yapı kurulmuş olabilir? **Cevap (öğrenci):** Birbirine yakın değerler tabloda birbirine yakın yerlere düşmesin, daha rastgele dağılsın diye.

Hoca bunun gerekçelerden biri olduğunu, ama bir gerekçe daha olduğunu söyledi. İkinci öğrenci "güvenlik için olabilir mi?" dedi ve hoca onayladı. Hoca bunu kriptografideki SHA yaklaşımına bağladı: Tek yönlü parola şifrelemede (one way password encryption) parola bu tür bir fonksiyondan geçirilip ağ üzerinden şifreli gider. Sunucu tarafında aynı fonksiyon çalıştırıldığında aynı değer üretiliyorsa parola eşleşmiş sayılır. Bu sırada kimse parolanın kendisini görmez.

Soru: Bunun geri dönüşü yok, değil mi? **Cevap:** Evet, geri dönüşü yok. Amaç güvenliği mümkün olduğunca yüksek bir yapı kurmaktır.

> **Ders sonrası not:** Burada iki ayrı konu karışmıştır. Evrensel hashingin amacı, fonksiyonu rastgele seçerek her farklı anahtar çifti için çakışma olasılığını en fazla $1/m$'de tutmaktır. Böylece kötü niyetli biri sabit bir fonksiyona göre hep aynı göze düşen girdiler hazırlayamaz ve hash tablosunun ortalama performansı garanti edilir. Parolaların saklanması ise SHA ailesi gibi kriptografik hash fonksiyonlarıyla (ve tuz/salt ile) yapılır; bu fonksiyonlar evrensel hashing ailesinden değildir. Evrensel hashingde p asal ve anahtar uzayından büyük, $a \in \{1, \dots, p-1\}$, $b \in \{0, \dots, p-1\}$ seçilir.

5. Yük Faktörü ve Ortalama Sınama Sayısı

Hoca kısa bilgilerle devam etti. Açık adresleme $m > n$ olduğu durumlarda, ayrık zincirleme $n > m$ olduğu durumlarda kullanılır. Tablonun doluluk oranını gösteren parametreye yük faktörü (load factor) denir:

$$\alpha = \frac{n}{m}$$

Bir hash yapısı kurulduğunda α'nın genelde 0,5 düzeyinde olması istenir.

5.1 Tablo: α'ya göre ortalama sınama sayısı

Hoca $\alpha = 0{,}1$'den $0{,}9$'a kadar ortalama sınama (average probe) sayılarını gösterdi. Sınama, deneme sayısıdır; yerleştirme ya da arama için kullanılabilir.

α	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9
Ortalama sınama	1,11	1,28	1,52	1,89	2,5	3,62	6,06	13	50,5

Hoca 0,7 değerini okumadı. Tablodaki 6,06 değeri aşağıdaki formülle hesaplanmıştır.

Bir öğrenci ne yapıldığını anlamadığını söyleyince hoca baştan anlattı: α, saklamak istediğiniz eleman sayısının tablo boyutuna oranıdır. Tablonun %10'u doluyken bir arama ya da ekleme yaklaşık bir adımda biter. 0,2 ve 0,3'te de durum benzerdir. 0,5'e gelindiğinde ortalama 2,5 adım sürer. %90 dolu bir tabloda ise 50 adımlık bir kontrol gerekir, çünkü büyük olasılıkla öbekler (cluster) oluşur ve bu öbekler yüzünden bu kadar deneme yapmak gerekir.

Soru: Bu tablo açık adresleme için mi, ayrık zincirleme için mi? **Cevap (öğrenci):** Açık adresleme, çünkü α 0 ile 1 arasında. Demek ki m, n'den büyük.

Hoca "aynen öyle, mantık basitti" dedi.

> **Ders sonrası not:** Tablodaki değerler doğrusal sınamada başarısız arama formülünden gelir: $U(\alpha) \approx \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$. Python ile hesaplanan değerler: 1,117; 1,281; 1,520; 1,889; 2,5; 3,625; 6,056; 13; 50,5. İlk değer yuvarlanınca 1,12'dir; derste 1,11 diye okundu.

6. Başarısız Aramada Ortalama Adım Sayısının Türetilmesi

Hoca başarısız arama (unsuccessful search) için ortalama adım sayısının nereden geldiğini gösterdi. Arama, boş bir göz görülene kadar sürer.

6.1 İlk gözün dolu olma olasılığı

İlk bakılan gözün dolu (occupied) olma olasılığına α diyelim. Boş olma olasılığı $1 - \alpha$'dır. Hoca önce bunun bir önceki α'dan farklı olduğunu söyledi ve öğrencilere bunun doğru olup olmadığını sordu.

Soru: Bu α, bir önceki α'dan farklı mı? **Cevap:** Bir öğrenci farklı olduğunu söyledi. Hoca şöyle sordu: Tablonun %10'u doluyken ilk baktığımız gözün dolu olma olasılığı da 0,1 değil midir? Öğrenci "doğru" dedi. Demek ki aynı α'dır. Hoca kimse itiraz etmeyince bunu bir kez daha vurgulamak için bilerek "farklı" dediğini söyledi.

6.2 İlk iki gözün dolu olma olasılığı

Soru: İlk iki gözün dolu olma olasılığı nedir? **Cevap (öğrenci):** %10'u doluyken, ilk gözün dolu çıktığını bildiğimize göre geriye 99 göz içinde 9 dolu göz kalır. İkinci gözün dolu olma olasılığı $9/99 = 1/11$ olur.

Hoca "kabaca söylemek gerekirse $\alpha \cdot \alpha$ diyebiliriz, o detayla çok uğraşmayalım" dedi. Yani ilk iki gözün dolu olma olasılığı yaklaşık α^2'dir. $\alpha = 0{,}1$ ise $0{,}1 \cdot 0{,}1$, $\alpha = 0{,}9$ ise $0{,}9 \cdot 0{,}9$ olur.

Öğrencinin hesabıyla (100 gözlü, 10 dolu tablo) kesin değer $0{,}1 \cdot \frac{9}{99} \approx 0{,}0091$, yaklaşık değer $\alpha^2 = 0{,}01$'dir (ara adım eklendi). Tablo büyüdükçe iki değer birbirine yaklaşır.

6.3 Aramanın tam k. adımda bitmesi

Arama ikinci adımda bitiyorsa ilk göz dolu, ikinci göz boştur: olasılık $\alpha(1-\alpha)$.

Bunu k adıma genelleyelim. Aramanın tam k. adımda bitmesi için ilk $k-1$ gözün hepsi dolu, k. göz boş olmalıdır:

$$P(k) = \alpha^{k-1}(1-\alpha)$$

6.4 Ortalama adım sayısı

Ortalama adım sayısı (average number of steps), her k değerinin olasılığıyla ağırlıklı toplamıdır:

$$E = \sum_{k=1}^{\infty} k\,\alpha^{k-1}(1-\alpha)$$

Hoca bunun geometrik seri açılımına götürdüğünü söyledi. Seri şöyle kapanır (ara adım eklendi):

$$\sum_{k=1}^{\infty} k\,\alpha^{k-1} = \frac{1}{(1-\alpha)^2} \quad \Rightarrow \quad E = (1-\alpha)\cdot\frac{1}{(1-\alpha)^2} = \frac{1}{1-\alpha}$$

> **Ders sonrası not:** Hoca serinin sonucunu "1 eksi alfa" diye söyledi; doğrusu $\frac{1}{1-\alpha}$'dır. Python ile $\alpha = 0{,}9$ için toplam 10 çıkar. Bu türetme her sınamanın birbirinden bağımsız olduğunu varsayar. Doğrusal sınamada öbeklenme olduğu için gerçek değer daha büyüktür; Bölüm 5'teki tablo ve Bölüm 7'deki formül bu yüzden $\alpha = 0{,}9$'da 10 değil 50,5 verir.
>
> **Ek bilgi (derste anlatılmadı):** $\frac{1}{1-\alpha}$ sonucu, her sınamanın tabloda rastgele ve bağımsız bir göze baktığı ideal modelin (uniform hashing) başarısız arama maliyetidir. Doğrusal sınamada dolu gözler art arda öbekler oluşturur. Bu olaya birincil öbeklenme (primary clustering) denir. Boş göze varmak için bir öbeğin tamamını geçmek gerektiğinden maliyet $\frac{1}{(1-\alpha)^2}$ mertebesine çıkar.

7. Doğrusal Sınamada Başarılı ve Başarısız Arama

Hoca sonra doğrusal sınamada başarılı (successful) ve başarısız (unsuccessful) arama formüllerini α üzerinden verdi. Bunların matematiksel analizini göstermek gerektiğini, ama vakit olmadığını ve gösteriminin biraz karışık olduğunu söyledi.

$$S(\alpha) \approx \frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right) \qquad U(\alpha) \approx \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$$

Hoca üç doluluk oranı için değerleri okudu:

α	Başarılı arama (S)	Başarısız arama (U)
0,50	1,5	2,5
0,75	2,5	8,5
0,90	5,5	50,5

α	Başarılı arama (S)	Başarısız arama (U)

Adım adım hesap (ara adım eklendi):

- $\alpha = 0{,}5$: $1 - \alpha = 0{,}5$. $S = \frac{1}{2}(1 + 2) = 1{,}5$. $U = \frac{1}{2}(1 + 4) = 2{,}5$.
- $\alpha = 0{,}75$: $1 - \alpha = 0{,}25$. $S = \frac{1}{2}(1 + 4) = 2{,}5$. $U = \frac{1}{2}(1 + 16) = 8{,}5$.
- $\alpha = 0{,}9$: $1 - \alpha = 0{,}1$. $S = \frac{1}{2}(1 + 10) = 5{,}5$. $U = \frac{1}{2}(1 + 100) = 50{,}5$.

Hoca 50,5 değerini dersin başındaki tabloda da gördüklerini hatırlattı. Python ile 10007 gözlü bir tabloda yapılan doğrusal sınama benzetimi de bu değerlere yakın sonuç verir (0,5: 1,50 ve 2,50; 0,75: 2,48 ve 8,58; 0,9: 5,76 ve 53,4).

7.1 Neden bu kadar büyük fark?

Soru: %90 dolulukta başarılı ve başarısız arama arasında neden bu kadar büyük fark var? **Cevap (öğrenci):** Doğru sonucu bulduğumuz anda işlemi kesiyoruz. Başarısız aramada ise her seferinde sona kadar gitmemiz gerekiyor.

Hoca cevabı açtı: Açık adreslemede başarısız aramada, tablo boyutu kadar demeyelim ama olabildiğince çok elemanın art arda gelme olasılığını düşünerek, boş göz (empty cell) görene kadar, örneğin 50 adım boyunca aramaya devam etmek zorundayız. Başarılı aramada ise tablo %90 dolu olsa bile arama büyük olasılıkla 5 adımda sonuçlanır. Art arda gelen öbekler birleştiğinde aradaki boş gözlerin sayısı çok azalır. Bu yüzden boş göze denk gelene kadar yapılan kontrol sayısı doğal olarak artar.

8. Ayrık Zincirlemede Arama Maliyeti

Hoca aynı soruyu ayrık zincirleme için sordu ve öğrencilere düşünmeleri için biraz süre verdi: Başarılı ve başarısız arama analizinden ne kadarlık bir karmaşıklık çıkar?

Bir öğrencinin fikri, düğümler arasında işlem yapıldığı için başarısız aramada oranın fazla olabileceğiydi. Hoca formülleri verdi:

- **Başarılı arama:** Bir kez hash fonksiyonu hesaplanır (1), artı ortalama $\alpha/2$ düğüm kontrol edilir: $1 + \dfrac{\alpha}{2}$.
- **Başarısız arama:** Hash hesabından sonra listenin bütün düğümleri, yani ortalama α düğüm kontrol edilir: $1 + \alpha$ (yaklaşık α).

8.1 n ve m örnekleri

Soru: Ayrık zincirleme olması için n ve m ne olabilir? Bir örnek verin. **Cevap (öğrenci):** Ayrık zincirlemede n, m'den büyük olacağı için α 1'den büyük çıkar. Örneğin $n = 2$, $m = 1$ alınırsa $\alpha = 2$ olur.

Hoca devam etti: Oran 3'e 1 olsaydı $\alpha = 3$ olurdu. Bu durumda ortalama 3 eleman aynı göze art arda yerleşir. Elemanlar aynı göze denk gelirse başarısız aramada bütün elemanları gezmek gerekir. Başarılı arama için hoca "$1 + \alpha/2$, $\alpha = 3$ ise 3 bölü 2'den 1,5" dedi.

> **Ders sonrası not:** $\alpha = 3$ için $\alpha/2 = 1{,}5$ yalnız liste kısmıdır. Formüldeki 1 de eklenince başarılı arama $1 + 1{,}5 = 2{,}5$ adım olur.

Hoca, en kötü durumu değil ortalama durumu (average case) düşündüklerini ve başarılı aramanın ayrık zincirlemede daha az adımda biteceğini vurguladı.

Hocanın örneği: $n = 100$, $m = 25$.

- $\alpha = 100/25 = 4$.
- Başarısız arama: hash hesabından sonra ortalama 4 düğüm gezilir ($1 + \alpha = 5$ adım, hash hesabı dahil).
- Başarılı arama: $1 + 4/2 = 1 + 2 = 3$ adım.

> **Ders sonrası not:** Hoca başarılı arama için "4 bölü 2'den 2, 3" dedi; doğru sonuç $1 + \alpha/2 = 3$'tür. Python ile 2000 kez rastgele 100 anahtar 25 gözlü tabloya başa eklenerek yerleştirildiğinde ortalama liste uzunluğu 4, başarılı aramada ortalama gezilen düğüm sayısı 2,98 çıkar.

Hoca örnekten şu sonucu çıkardı: n büyük olsa bile, 100 eleman ve 25 gözlük bir tabloda, 100/25'ten ortalama 4 elemanın art arda gelme olasılığı vardır. Teoride 100 elemanın hepsinin aynı göze düşme ihtimali görünse de pratikte bu olmaz. Bu yüzden $n > m$ olduğu durumlarda ayrık zincirlemeden yararlanmak akıllıcadır.

> **Ek bilgi (derste anlatılmadı):** Başarılı arama için kitaplarda verilen daha kesin ifade $1 + \frac{\alpha}{2} - \frac{1}{2m}$'dir; m büyükken $1 + \alpha/2$'ye eşit kabul edilir. Bu formüller, her anahtarın m gözden birine eşit olasılıkla ve diğerlerinden bağımsız düştüğü varsayımına (simple uniform hashing) dayanır. Ayrık zincirlemede α'nın 1'den büyük olabilmesi, açık adreslemeden temel farkıdır.

9. Doğrusal ve İkinci Dereceden Sınamanın Karşılaştırılması

Hoca son olarak tahtaya bir grafik çizdi. Yatay eksende α vardı: 0,2'den başlayıp 0,4, 0,6, 0,8 ve 1'e kadar. Dikey eksende ortalama sınama sayısı (average number of probes) vardı: 1'den başlayıp 1,5, 2 diye yukarı doğru gidiyordu. Grafikte iki eğri vardı ve grafik başarılı aramaya aitti.

Soru: Bu iki eğri derste gördüğümüz hangi yöntemlere ait olabilir? **Cevap (birinci öğrenci):** Kırmızı açık adresleme, mavi ayrık zincirleme.

Hoca bu ikisinin aynı grafikte gösterilemeyeceğini söyledi, çünkü grafiğin yatay ekseni 1 ile sınırlıdır. α 1'e kadar sınırlı olduğuna göre iki eğri de açık adresleme yöntemlerine aittir. Hoca denemenin iyi olduğunu da ekledi.

Cevap (ikinci öğrenci): Emin olmamakla birlikte soldaki (kırmızı, daha hızlı yükselen) doğrusal sınama olabilir. Sağdaki de ikinci dereceden sınama ya da çift hashing olabilir.

Hoca "aynen öyle" dedi. İkisi de olabilir, ama verdiği örnek ikinci dereceden sınamada başarılı aramanın kaç adımda tamamlandığını gösteriyordu. Grafikte aynı doluluk oranında bir kesit alındığında, doğrusal sınama başarılı arama için daha fazla deneme gerektirir. İkinci dereceden sınama deneme sayısını azaltır.

> **Hocanın vurgusu:** Aynı doluluk oranındaki iki tabloda ikinci dereceden sınama, arama işlemlerini doğrusal sınamadan daha az adımda gerçekleştirir. Bu yüzden ikinci dereceden sınama doğrusal sınamaya göre daha avantajlıdır. Önemli noktalardan biri budur.

> **Ek bilgi (derste anlatılmadı):** İkinci dereceden sınamada sınama adımları $1, 4, 9, \ldots$ diye büyüdüğü için aynı öbeğin içinde tek tek ilerlenmez ve birincil öbeklenme büyük ölçüde ortadan kalkar. Yine de aynı ilk adrese düşen anahtarlar aynı sınama dizisini izler; buna ikincil öbeklenme (secondary clustering) denir. Çift hashing bu sorunu da azaltır.

10. Ders Sonu Soruları

Hoca hash konusunu bitirdikten sonra öğrencilerden geriye dönük sorular aldı. Akademik olanlar şunlardır.

Soru: Açık adresleme ve ayrık zincirleme yalnız bu durumlarda mı kullanılabilir? **Cevap:** Slayttaki durumlar, kullanılması mantıklı olan durumları gösterir; bir şart değildir. $m > n$ ise açık adresleme tavsiye edilir, ama ayrık zincirleme de kullanılabilir. $n > m$ ise açık adreslemeyi kullanmak genelde mümkün olmaz, çünkü tablo büyük olasılıkla dolar ve hata alınır. Yani açık adreslemenin alternatifi ayrık zincirleme olabilir. Ama $n > m$ iken ayrık zincirlemenin alternatifi açık adresleme olamayabilir.

Soru: Bu örneklerde mod 6'ya göre mod aldık. Oysa tablo boyutunun hep asal sayı olması gerektiğini söylemiştiniz. **Cevap:** 6, tamamen kolaylık için seçilmiş bir sayıdır. Tercih edilen, tablo boyutunun asal sayı (prime number) olmasıdır.

Hocanın vurgusu: Gerçek uygulamada tablo boyutu m asal seçilmelidir. Derste $m = 6$ yalnızca örneği kolaylaştırmak için kullanıldı.

Örnek Sorular

Bu derste ekran kaydı ya da kare yoktur. Aşağıdaki sorular derste sözlü sorulan sorulardır.

1. (derste sözlü, 00:02:47) $m = 6$ ve $h(k) = k \bmod 6$ olan ayrık zincirlemeli bir tabloya önce 13, sonra 1 yerleştirildi. 19 geldiğinde nereye yerleşir?

Çözüm: $19 \bmod 6 = 1$. 19, 1 numaralı gözün listesine girer. Listenin sonuna gitmek gereksiz maliyet olduğu için başa eklenir.

2. (derste sözlü, 00:05:16) Ayrık zincirlemeyi tablo boyutu m ile eleman sayısı n arasında hangi ilişkide kullanmak akıllıca olur?

Çözüm: $n > m$ iken. Açık adresleme ise $m > n$ iken tavsiye edilir.

3. (derste sözlü, 00:06:04) Ayrık zincirleme kullanan hashlemenin karmaşıklığı nedir?

Çözüm: Teorik en kötü durum, bütün elemanlar aynı göze düşerse $O(n)$'dir. Pratikte iyi bir hash fonksiyonu ve tablo boyutuyla $O(1)$ kabul edilir; ortalama maliyet $\Theta(1 + \alpha)$'dır.

4. (derste sözlü, 00:08:37 ve 00:09:46) `struct node` ve `struct hash` içinde hangi bilgiler saklanmalıdır?

Çözüm: `struct node`: anahtar (`key`), değer (`name`) ve sonraki düğümün işaretçisi (`next`). `struct hash`: listenin başı (`head`) ve o gözdeki düğüm sayısı (`count`).

5. (derste sözlü, 00:13:18) `insertToHash` yazarken hangi adımlardan geçilir?

Çözüm: `hashIndex = key % M` hesaplanır, `createNode` ile düğüm oluşturulur. Göz boşsa düğüm `head` olur ve `count = 1`. Göz doluysa düğüm listenin başına eklenir ve `count` bir artırılır.

6. (derste sözlü, 00:21:25) Sona eklemeli bir yapı ve tek bağlı liste kullanılsaydı hangi bilgiyi saklamak akıllıca olurdu?

Çözüm: Listenin son elemanının adresi, yani `tail`.

7. (derste sözlü, 00:25:52) `searchInHash` fonksiyonunda hangi adımlar izlenir?

Çözüm: Hash indeksi hesaplanır, o gözün `head`'inden başlanır. Düğüm NULL değilken ve anahtar eşleşmezken `next` ile ilerlenir. Döngüden çıkınca bayrağa bakılır; set edilmediyse "eleman bulunamadı" mesajı verilir.

8. (derste sözlü, 00:32:47) Evrensel hashingde a, b, p neden rastgele seçilir? Ayrıca $a = 3$, $b = 4$, $p = 17$, $m = 6$ için $k = 8$ hangi göze düşer?

Çözüm: $(3 \cdot 8 + 4) \bmod 17 = 28 \bmod 17 = 11$, $11 \bmod 6 = 5$. Derste gerekçe olarak yakın değerlerin tabloda dağılması ve güvenlik söylendi. Asıl amaç, girdiden bağımsız olarak çakışma olasılığını ortalamada düşük tutmaktır (Bölüm 4.2'deki ders sonrası nota bakın).

9. (derste sözlü, 00:37:49) α'ya göre ortalama sınama sayısı tablosu (1,11; 1,28; ...; 50,5) açık adresleme için mi, ayrık zincirleme için mi?

Çözüm: Açık adresleme için. α 0 ile 1 arasında verildiğine göre $m > n$'dir.

10. (derste sözlü, 00:41:54) İlk iki bakılan gözün dolu olma olasılığı nedir? Aramanın tam k. adımda bitme olasılığı ve ortalama adım sayısı nedir?

Çözüm: Yaklaşık α^2. $P(k) = \alpha^{k-1}(1 - \alpha)$. Ortalama $\sum_{k \geq 1} k\,\alpha^{k-1}(1 - \alpha) = \frac{1}{1-\alpha}$ (bağımsız sınama varsayımıyla).

11. (derste sözlü, 00:48:03) %90 dolu bir tabloda doğrusal sınamada başarılı arama 5,5, başarısız arama 50,5 sınama sürüyor. Fark neden bu kadar büyük?

Çözüm: Başarılı arama anahtar bulununca durur. Başarısız arama boş göz görülene kadar sürer. Tablo dolunca öbekler birleşir, boş göz azalır ve boş göze varmak için uzun öbekler geçilir.

12. (derste sözlü, 00:52:04 ve 00:53:54) Ayrık zincirlemede $n = 100$, $m = 25$ için başarılı ve başarısız aramada ortalama kaç adım gerekir?

Çözüm: $\alpha = 4$. Başarılı arama: $1 + 4/2 = 3$. Başarısız arama: hash hesabından sonra ortalama 4 düğüm, hash hesabı dahil $1 + 4 = 5$.

13. (derste sözlü, 00:56:06) Başarılı arama için α ekseni 1'e kadar olan bir grafikte iki eğri var. Bunlar hangi yöntemlere aittir?

Çözüm: Eksen 1 ile sınırlı olduğu için ikisi de açık adresleme yöntemidir. Daha hızlı yükselen eğri doğrusal sınama, diğeri ikinci dereceden sınamadır (derste örnek ikinci dereceden sınama için verildi; çift hashing de benzer davranır).

Dersin Toparlaması

Bu derste hash tablolarında çakışma çözümünün ikinci yolu olan ayrık zincirleme tamamlandı. Her göz bir bağlı listenin başını tutar; aynı adrese düşen anahtarlar bu listeye, maliyeti düşük tutmak için başa eklenir. Ayrık zincirleme $n > m$ iken tercih edilir. Teorik en kötü durumu $O(n)$ olsa da iyi bir hash fonksiyonu ve tablo boyutuyla ortalama maliyeti $1 + \alpha/2$ (başarılı) ve $1+\alpha$ (başarısız) olur. Ekleme, düğüm oluşturma ve arama C ile adım adım kuruldu; silme öğrencilere bırakıldı. Ardından evrensel hashingde fonksiyonun $h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$ ailesinden rastgele seçildiği görüldü.

Dersin ikinci yarısı yük faktörü $\alpha = n/m$ ile arama maliyeti arasındaki ilişkiye ayrıldı. Bağımsız sınama varsayımıyla başarısız aramanın ortalama $\frac{1}{1-\alpha}$ adım sürdüğü geometrik seriyle türetildi. Doğrusal sınamada öbeklenme yüzünden bu değer çok daha büyüktür: %90 dolulukta başarılı arama 5,5, başarısız arama 50,5 sınama sürer. Bu yüzden α'nın 0,5 civarında tutulması istenir. Son olarak ikinci dereceden sınamanın aynı dolulukta doğrusal sınamadan daha az sınama ile arama yaptığı gösterildi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.